

Grocery Delivery Platform

A production-style DevOps platform for grocery delivery operations — automated deployments, autoscaling, monitoring and resilient, fully self-hosted infrastructure.

GitHub + Git

FastAPI

PostgreSQL

Docker, docker-compose

Jenkins

Terraform

Kubernetes on kind + HPA

Prometheus + Grafana

ELK

HashiCorp Vault

NAME

Tejas Mhatre

ROLL NO

150096724085

COHORT

Elon Musk

SUBJECT

DevOps

Problem Statement

Industry: E-Commerce

The Grocery Delivery Platform is experiencing **order processing delays**, **infrastructure scalability issues**, **deployment bottlenecks**, and **insufficient monitoring capabilities** during peak demand periods.

Our Solution

We designed and built a complete, production-style DevOps platform that runs entirely on **local / self-hosted infrastructure** (Docker, kind, Terraform local provider) — no cloud-managed services. Each concern in the problem statement is addressed directly:

- **Order processing delays** → FastAPI backend with an indexed PostgreSQL store and an explicit order state machine.
- **Scalability issues** → Kubernetes with a Horizontal Pod Autoscaler that scales the app 2→10 pods automatically under load.
- **Deployment bottlenecks** → a Jenkins CI/CD pipeline (build, test, image, push, deploy) and reproducible Terraform infrastructure.
- **Poor monitoring** → Prometheus + Grafana metrics, the ELK stack for logs, alert rules, and a Disaster Recovery plan.
- **Secrets** → HashiCorp Vault for database credentials and API keys, injected into the app at startup.

Technology Stack

Source / VCS	GitHub + Git
Application	FastAPI (Python), SQLAlchemy
Database	PostgreSQL (SQLite for dev)
Containerization	Docker, docker-compose
CI / CD	Jenkins (declarative pipeline)
Infrastructure as Code	Terraform (local Docker provider)
Orchestration	Kubernetes on kind + HPA
Monitoring	Prometheus + Grafana
Logging	ELK (Elasticsearch, Logstash, Kibana)

1. Application — FastAPI

Backend with endpoints for placing orders, tracking status, inventory check and delivery assignment. SQLAlchemy models, an order state machine, and a built-in dashboard. Validated with an automated test suite.

```
pytest tests/

.venv/lib/python3.9/site-packages/fastapi/applications.py:4495
/Users/tejasmathre/Desktop/grocery/.venv/lib/python3.9/site-packages/fastapi/applications.py:4495: DeprecationWarning:
    on_event is deprecated, use lifespan event handlers instead.

    Read more about it in the
    [FastAPI docs for Lifespan Events](https://fastapi.tiangolo.com/advanced/events/).

    return self.router.on_event(event_type)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
6 passed, 7 warnings in 1.01s
```

Automated tests — 6 passing

2. Docker — Containerization

Multi-stage Dockerfile (non-root, healthcheck) and docker-compose running the backend together with PostgreSQL. The backend also serves the dashboard.

Grocery Delivery Platform

Order processing · inventory · delivery assignment — DevOps demo dashboard

Inventory

SKU	Name	Price	Stock	Add
BANANA-1KG	Bananas 1kg	\$1.10	90	Add
BREAD-WHT	White Bread	\$0.95	150	Add
COFFEE-250G	Coffee 250g	\$4.20	59	Add
EGGS-12	Eggs (dozen)	\$2.50	119	Add
MILK-1L	Milk 1L	\$1.20	198	Add
OIL-1L	Cooking Oil 1L	\$3.30	98	Add
RICE-5KG	Rice 5kg	\$6.80	79	Add
TOMATO-1KG	Tomatoes 1kg	\$1.60	70	Add

Place an order

Customer

Jane Doe

Address

12 Market St

Click "Add" on inventory rows to build a cart.

Place order

Orders

ID	Customer	Status	Total	Agent	Actions
3	Meera S	CONFIRMED	\$4.20	—	Assign
2	Raj Kumar	ASSIGNED	\$13.40	Asha	Dispatch
1	Jane Doe	DELIVERED	\$4.90	Sana	

Dashboard served from the container (live orders)

Grocery Delivery Platform

1.0.0

OAS 3.1

/openapi.json

ops

GET /health Health

GET /ready Ready

GET /metrics Metrics

inventory

GET /api/inventory Inventory

GET /api/inventory/{sku} Inventory Item

orders

GET /api/orders List Orders

POST /api/orders Place Order

GET /api/orders/{order_id} Track Order

PATCH /api/orders/{order_id}/status Update Order Status

delivery

POST /api/orders/{order_id}/assign Assign Delivery

Swagger / OpenAPI documentation

```
docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
grocery-app	grocery-app:local	"uvicorn app.main:ap..."	app	3 minutes ago	Up 3 minutes (healthy)	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
grocery-db	postgres:16-alpine	"docker-entrypoint.s..."	db	3 minutes ago	Up 3 minutes (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp

docker compose ps — app + database healthy

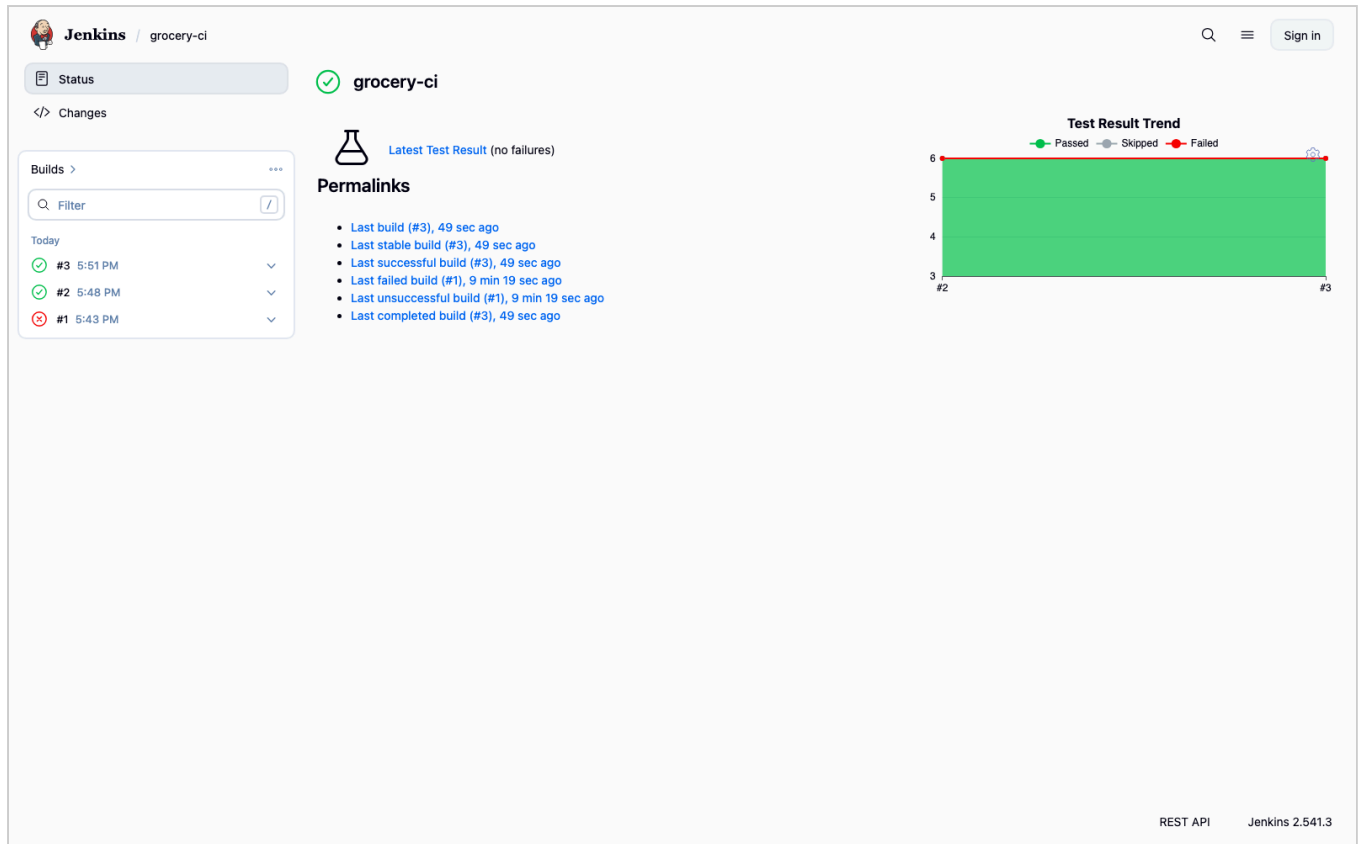
```
docker images
```

NAME	IMAGE	SIZE	REPOSITORY	PLATFORM
grocery-app:local	b5d3be087d77	425MB	103MB	U
postgres:16-alpine	e013e867e712	411MB	115MB	U

Built images

3. Jenkins — CI/CD Pipeline

Declarative pipeline: Checkout, Build & Test, Docker Build, Push to registry, Deploy. Tests run in an isolated python:3.11 container; the image is pushed to a local registry.



Pipeline builds green; 6 tests passing

```
grocery-ci #3 - all stages SUCCESS

STAGE ▶ Checkout
STAGE ▶ Build & Test
        6 passed, 7 warnings in 0.84s
STAGE ▶ Docker Build
        Successfully tagged localhost:5051/grocery-app:3
        Successfully tagged localhost:5051/grocery-app:latest
STAGE ▶ Push to Registry
        + docker push localhost:5051/grocery-app:3
        + docker push localhost:5051/grocery-app:latest
STAGE ▶ Deploy
        Deploy: kubectl set image deployment/grocery-app on the kind cluster (requires a kub
Finished: SUCCESS
```

Console — all stages SUCCESS

Jenkins

grocery-ci

#3

Status

Changes

Console Output

View Build Information

Git Build Data

Tests

Pipeline Steps

Workspaces

Previous Build

✓ #3 (Jun 20, 2026, 5:51:56 PM)

Started by user [admin](#)

Revision: 7069d2996e7e94e0849559f096c78f4c1cfb5570

Repository: https://github.com/Tejas75060/grocery_devops.git

• refs/remotes/origin/main

Tests (no failures)

No changes.

Started 56 sec ago

Took [22 sec](#)

REST API

Jenkins 2.541.3

Build detail — git revision + tests

Jenkins

Build History

Build Queue

Build Executor Status

Grocery Delivery Platform — CI/CD (configured via JCasC)

All

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☀	grocery-ci	3 min 28 sec #3	11 min #1	22 sec

Icon: S M **L**

REST API

Jenkins 2.541.3

Jenkins dashboard

4. Terraform — Infrastructure as Code

Provisions the local environment (network, PostgreSQL, app, registry) with the local Docker provider — no cloud-managed services. Local state backend.

```
terraform apply

$ terraform apply -auto-approve
...
docker_container.app: Creation complete
docker_container.db: Creation complete
docker_container.registry: Creation complete

Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:
app_url = "http://localhost:8000"
database_container = "grocery-db"
registry_url = "localhost:5055"
```

terraform apply — 8 resources created

```
terraform output

app_url = "http://localhost:8000"
database_container = "grocery-db"
registry_url = "localhost:5055"
```

terraform output

```
docker ps - terraform-provisioned containers
```

NAMES	IMAGE	STATUS	PORTS
grocery-registry	a3d8aaa63ed8	Up 34 seconds	0.0.0.0:5055->5000/tcp
grocery-app	aeedd18f1688	Up 38 seconds (healthy)	0.0.0.0:8000->8000/tcp
grocery-db	e013e867e712	Up 48 seconds (healthy)	5432/tcp
grocery-control-plane	kindest/node:v1.30.0	Up 36 minutes	0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp,
127.0.0.1:64662->6443/tcp			
grocery-worker	kindest/node:v1.30.0	Up 36 minutes	

Provisioned containers

5. Kubernetes — Orchestration & Autoscaling

Deployment, Service, Ingress, ConfigMap, Secret, a Postgres StatefulSet, and an HPA on a local kind cluster. The HPA scales the app under load.

```
kubectl -n grocery get pods,svc,ingress,hpa
```

NAME	READY	STATUS	RESTARTS	AGE
pod/grocery-app-555c98f7b9-76r5r	1/1	Running	0	2m54s
pod/grocery-app-555c98f7b9-b8r8c	1/1	Running	0	2m54s
pod/grocery-db-0	1/1	Running	0	3m30s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/grocery-app	ClusterIP	10.96.99.75	<none>	80/TCP	2m54s
service/grocery-db	ClusterIP	None	<none>	5432/TCP	3m30s

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
ingress.networking.k8s.io/grocery-ingress	nginx	grocery.localhost	localhost	80	2m54s

NAME	REFERENCE	TARGETS	MINPODS
MAXPODS REPLICAS AGE			
horizontalpodautoscaler.autoscaling/grocery-app-hpa	Deployment/grocery-app	cpu: 11%/60%, memory: 60%/75%	2 10
2 2m54s			

pods, services, ingress, HPA

Grocery Delivery Platform

Order processing · Inventory · delivery assignment — DevOps demo dashboard

Inventory

SKU	Name	Price	Stock	Add
BANANA-1KG	Bananas 1kg	\$1.10	87	Add
BREAD-WHT	White Bread	\$0.95	150	Add
COFFEE-250G	Coffee 250g	\$4.20	60	Add
EGGS-12	Eggs (dozen)	\$2.50	120	Add
MILK-1L	Milk 1L	\$1.20	200	Add
OIL-1L	Cooking Oil 1L	\$3.30	100	Add
RICE-5KG	Rice 5kg	\$6.80	80	Add
TOMATO-1KG	Tomatoes 1kg	\$1.60	70	Add

Place an order

Customer

Jane Doe

Address

12 Market St

Click "Add" on inventory rows to build a cart.

Place order

Orders

ID	Customer	Status	Total	Agent	Actions
1	K8s User	CONFIRMED	\$3.30	—	Assign

App via Ingress (grocery.localhost)

```
$ kubectl -n grocery get hpa -w (during load test)

10%/60%, memory:    cpu/mem: 60%/75%   replicas: 2
158%/60%, memory:   cpu/mem: 61%/75%   replicas: 2
454%/60%, memory:   cpu/mem: 61%/75%   replicas: 2
457%/60%, memory:   cpu/mem: 54%/75%   replicas: 4
379%/60%, memory:   cpu/mem: 54%/75%   replicas: 4
356%/60%, memory:   cpu/mem: 50%/75%   replicas: 6
299%/60%, memory:   cpu/mem: 50%/75%   replicas: 6
294%/60%, memory:   cpu/mem: 50%/75%   replicas: 8
263%/60%, memory:   cpu/mem: 50%/75%   replicas: 8
209%/60%, memory:   cpu/mem: 49%/75%   replicas: 10
```

HPA autoscaling 2 to 8 pods under load

kubectl -n grocery get hpa,pods (scaled to 8)

NAME	REFERENCE	TARGETS	MINPODS
MAXPODS REPLICAS AGE			
horizontalpodautoscaler.autoscaling/grocery-app-hpa	Deployment/grocery-app	cpu: 209%/60%, memory: 49%/75%	2 10
10 5m40s			
NAME	READY	STATUS	RESTARTS AGE
pod/grocery-app-555c98f7b9-76r5r	1/1	Running	0 5m40s
pod/grocery-app-555c98f7b9-9tfs8	1/1	Running	0 84s
pod/grocery-app-555c98f7b9-b8r8c	1/1	Running	0 5m40s
pod/grocery-app-555c98f7b9-g6xxg	1/1	Running	0 54s
pod/grocery-app-555c98f7b9-ng5f7	1/1	Running	0 114s
pod/grocery-app-555c98f7b9-q7g28	1/1	Running	0 84s
pod/grocery-app-555c98f7b9-rklf2	1/1	Running	0 23s
pod/grocery-app-555c98f7b9-s7gmr	1/1	Running	0 23s
pod/grocery-app-555c98f7b9-szkh7	1/1	Running	0 54s
pod/grocery-app-555c98f7b9-wtxr2	1/1	Running	0 114s

Scaled replicas + HPA status

6. Monitoring & Logging — Prometheus, Grafana, ELK

Prometheus scrapes app metrics via Kubernetes service discovery; Grafana visualises them. The ELK stack centralises structured application logs.

Prometheus Alerts Graph Status ▾ Help

Targets

All scrape pools ▾

All Unhealthy Collapse All

Filter by endpoint or labels

Unknown

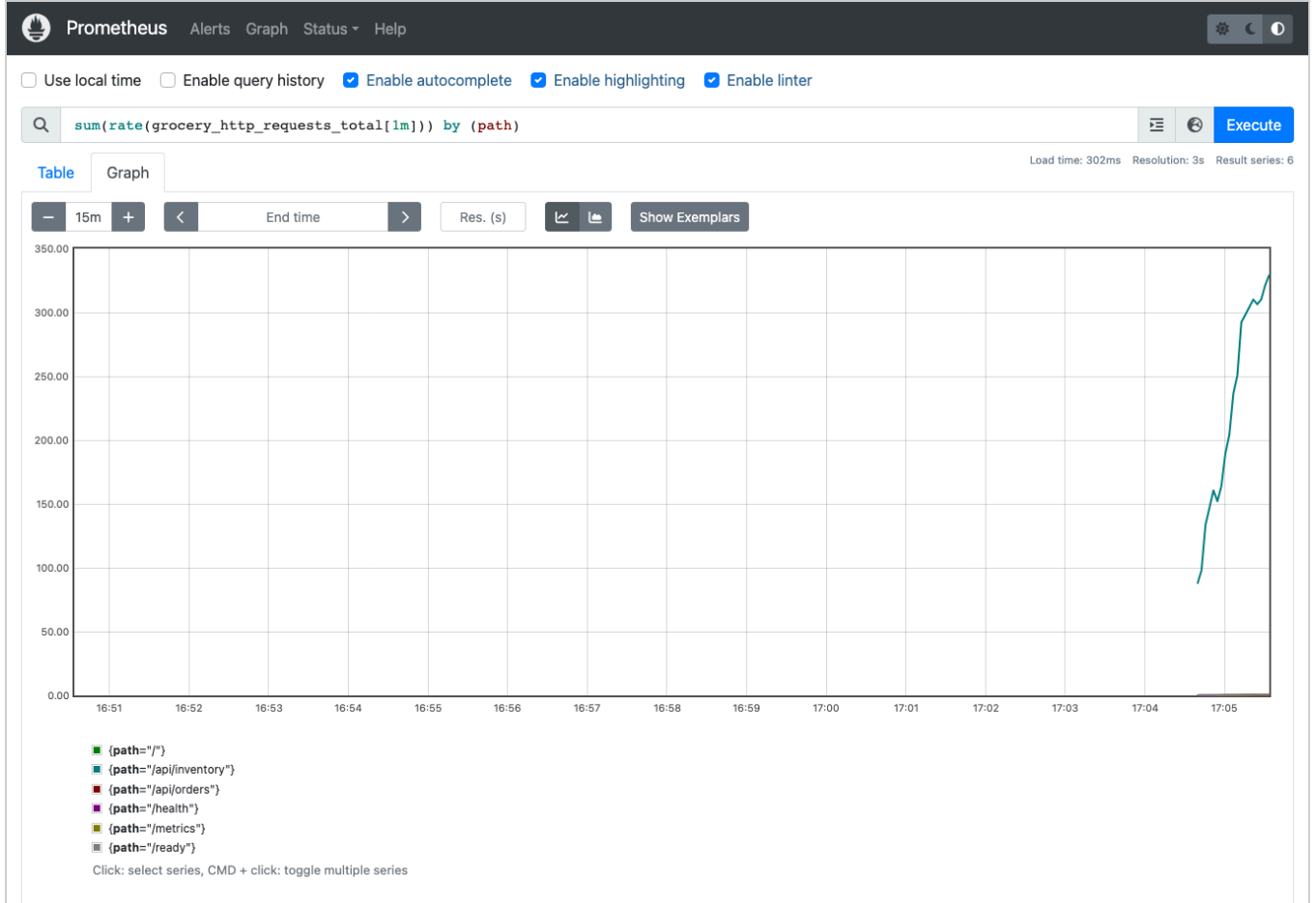
Unhealthy

Healthy

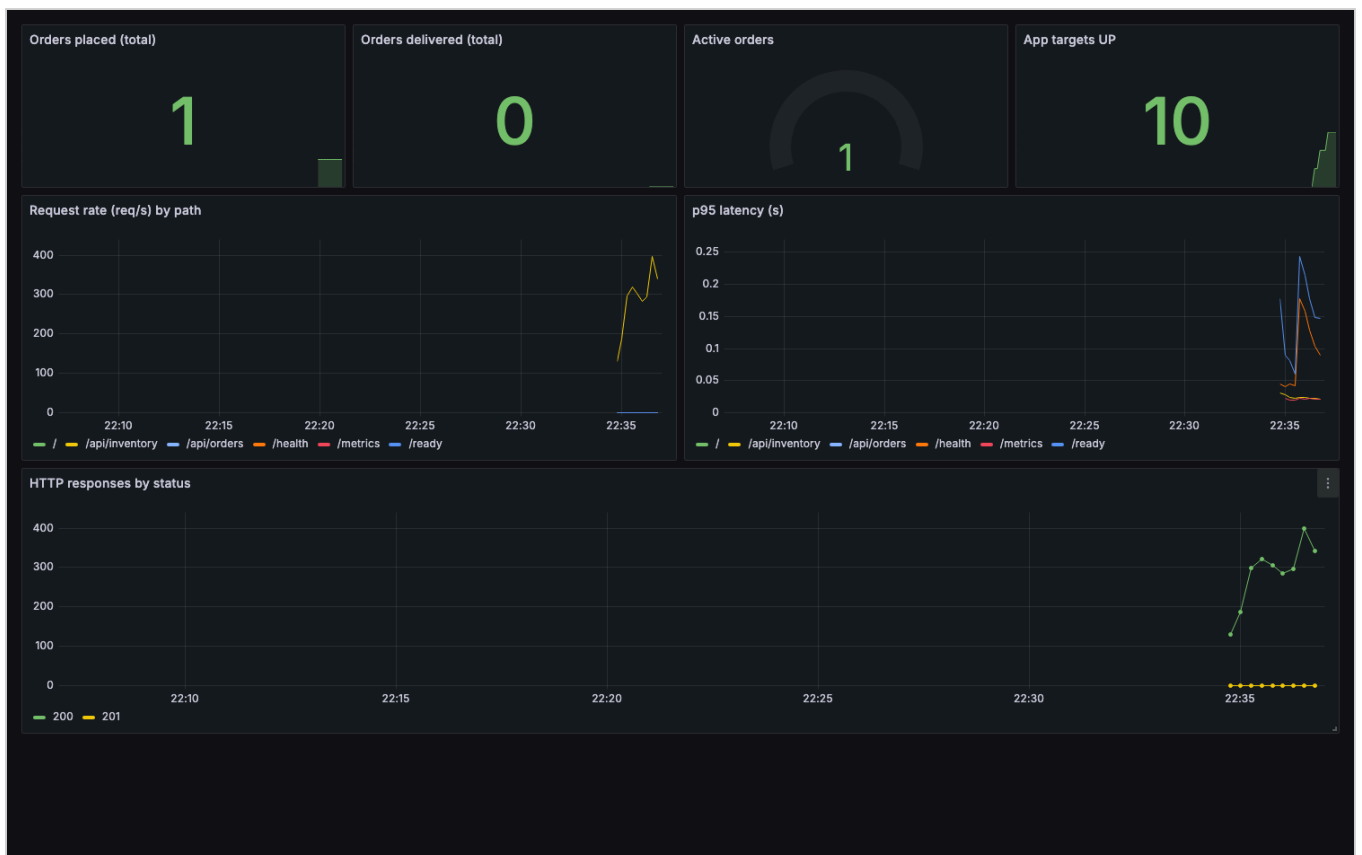
grocery-app (8/8 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://10.244.1.19:8000/metrics	UP	instance="10.244.1.19:8000" job="grocery-app" namespace="grocery" pod="grocery-app-555c98f7b9-9tfs8" ▾	661.000ms ago	25.032ms	
http://10.244.1.22:8000/metrics	UP	instance="10.244.1.22:8000" job="grocery-app" namespace="grocery" pod="grocery-app-555c98f7b9-szkh7" ▾	11.308s ago	43.624ms	
http://10.244.1.21:8000/metrics	UP	instance="10.244.1.21:8000" job="grocery-app" namespace="grocery" pod="grocery-app-555c98f7b9-g6xxg" ▾	13.580s ago	49.147ms	
http://10.244.1.9:8000/metrics	UP	instance="10.244.1.9:8000" job="grocery-app" namespace="grocery" pod="grocery-app-555c98f7b9-76r5r" ▾	4.503s ago	84.976ms	
http://10.244.1.10:8000/metrics	UP	instance="10.244.1.10:8000" job="grocery-app" namespace="grocery" pod="grocery-app-555c98f7b9-b8r8c" ▾	4.381s ago	11.758ms	
http://10.244.1.18:8000/metrics	UP	instance="10.244.1.18:8000" job="grocery-app" namespace="grocery" pod="grocery-app-555c98f7b9-ng5f7" ▾	12.768s ago	7.110ms	
http://10.244.1.17:8000/metrics	UP	instance="10.244.1.17:8000" job="grocery-app" namespace="grocery" pod="grocery-app-555c98f7b9-wtxr2" ▾	3.599s ago	10.540ms	
http://10.244.1.20:8000/metrics	UP	instance="10.244.1.20:8000" job="grocery-app" namespace="grocery" pod="grocery-app-555c98f7b9-n7n28" ▾	11.437s ago	23.982ms	

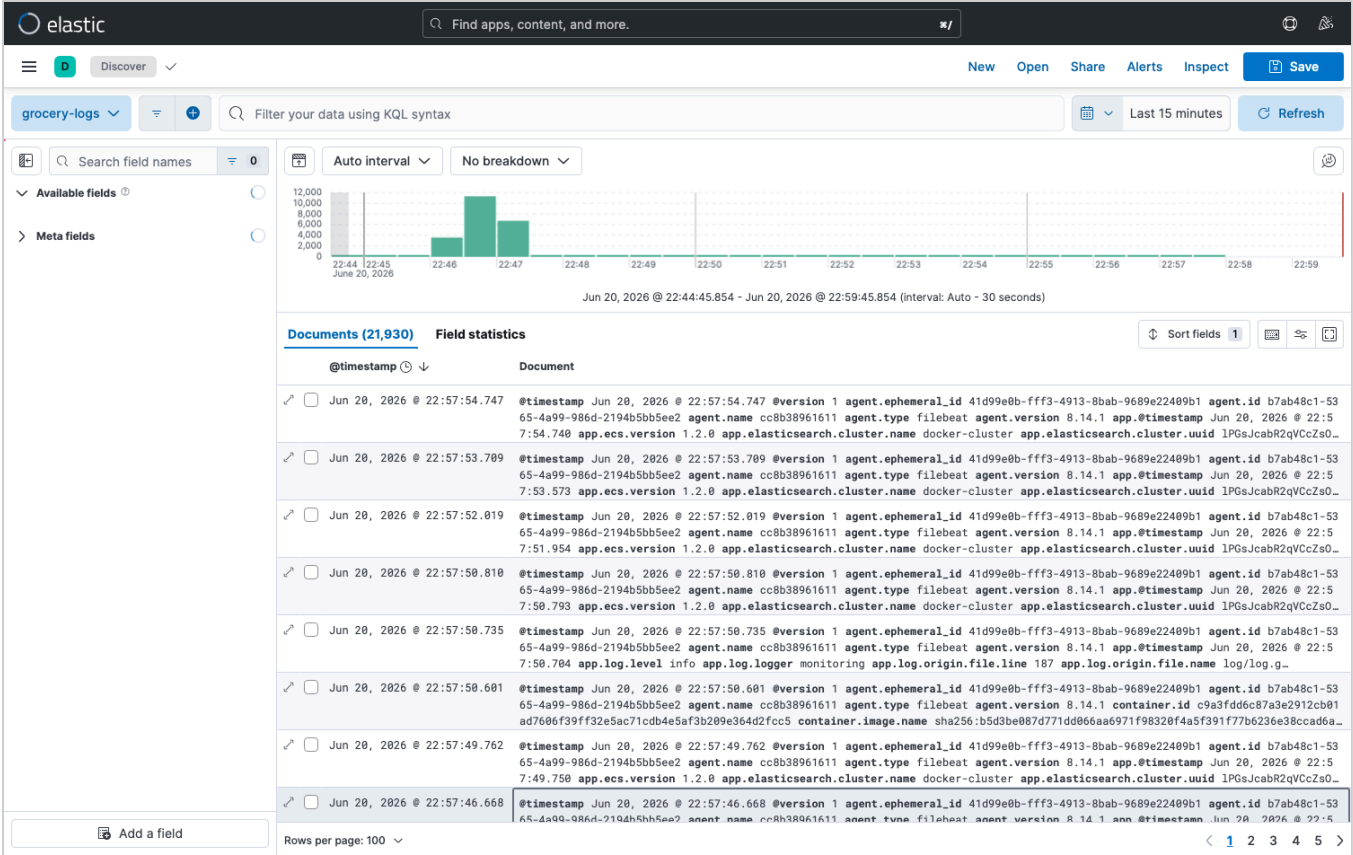
Prometheus targets UP



Prometheus — request-rate query



Grafana dashboard (live data)



Kibana Discover — grocery logs

Elasticsearch: grocery-logs-* index

health	status	index	uuid	pri	rep	docs.count	docs.deleted	store.size	pri.store.size
yellow	open	grocery-logs-2026.06.20	rooThf0yQ2mhJ9olvgiY8g	1	1	2753	0	3.5mb	3.5mb

Elasticsearch log index

7. Vault — Secrets Management

HashiCorp Vault stores DB credentials / API keys. The app loads its database URL from Vault at startup instead of from plain config.

```
vault kv get secret/grocery

=== Secret Path ===
secret/data/grocery

===== Metadata =====
Key          Value
-----
created_time 2026-06-20T16:59:22.124974179Z
custom_metadata <nil>
deletion_time n/a
destroyed     false
version       2

===== Data =====
Key          Value
-----
api_key      demo-api-key-123
database_url  postgresql://grocery:grocery@localhost:5432/grocery
db_password  grocery
db_user      grocery
```

Secret stored in Vault (KV v2)

```
app reads DB creds from Vault

$ # WITHOUT Vault
database_url = sqlite:///./grocery.db

$ # WITH Vault env (VAULT_ADDR/VAULT_TOKEN set)
INFO:grocery.config:Loaded database_url from Vault
database_url = postgresql://grocery:grocery@localhost:5432/grocery
```

App loads DB credentials from Vault



Sign in to Vault

Method

Token

Token

Sign in

Contact your administrator for login credentials.

[Vault](#)

[Upgrade to Vault Enterprise](#)

[Documentation](#)

[Support](#)

[Terms](#)

[Privacy](#)

[Security](#)

[Accessibility](#)

 © 2026 HashiCorp

